

Observation Task 2: Bank Account Management System using Classes and Objects (CO1)

Observation Task: 02

Date: 25/09/2026

■ Objective / Aim

To design and implement a **Bank Account Management System** in Java using **classes and objects**, demonstrating object-oriented programming concepts such as attributes, methods, constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, and fund transfers.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

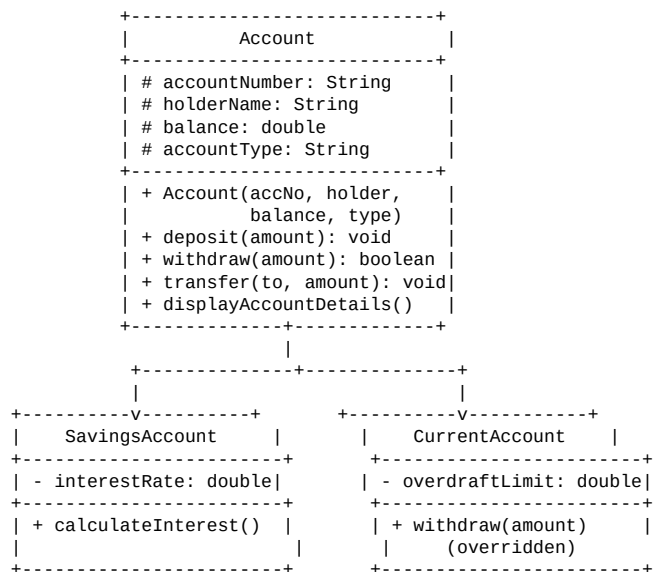
■ Algorithm / Logic Description

1. Create a base class named **Account**.
2. Define the following attributes in the Account class:
 - Account number
 - Account holder name
 - Balance
 - Account type
3. Define a constructor to initialize the account details.
4. Define the following methods in the Account class:
 - deposit() to add money to the account.
 - withdraw() to withdraw money when sufficient balance is available.
 - transfer() to transfer money from one account to another.
 - displayAccountDetails() to display account information.
5. Create a **SavingsAccount** class that inherits from Account.
6. Add interestRate as an additional attribute in SavingsAccount.
7. Define calculateInterest() to calculate interest based on the account balance.
8. Create a **CurrentAccount** class that inherits from Account.
9. Add overdraftLimit as an additional attribute in CurrentAccount.

10. Override the withdraw() method in CurrentAccount to allow withdrawal up to the permitted overdraft limit.
11. Create objects for SavingsAccount and CurrentAccount.
12. Perform deposits, withdrawals, interest calculation, and fund transfer operations.
13. Display the account details before and after the transactions.
14. Display the final balance of both accounts.

Note: SavingsAccount and CurrentAccount inherit from the Account class.

UML Class Diagram



*Relationship: SavingsAccount and CurrentAccount **inherit** from the Account class. CurrentAccount **overrides** withdraw() to support overdraft-based withdrawals, demonstrating both inheritance and method overriding.*

Source Code

```

class Account {
    protected String accountNumber;
    protected String holderName;
    protected double balance;
    protected String accountType;

    // Constructor to initialize account details
    Account(String accountNumber, String holderName,
            double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
        this.accountType = accountType;
    }

    // Deposit money into the account
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: Rs." + amount);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    // Withdraw money from the account
    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
        }
    }
}
  
```

```

        System.out.println("Withdrawn: Rs." + amount);
        return true;
    }
    System.out.println("Insufficient balance.");
    return false;
}

// Transfer money to another account
public void transfer(Account target, double amount) {
    if (withdraw(amount)) {
        target.deposit(amount);
        System.out.println("Transferred: Rs." + amount
            + " to " + target.accountNumber);
    } else {
        System.out.println("Transfer failed.");
    }
}

// Display account details
public void displayAccountDetails() {
    System.out.println("Account Number : " + accountNumber);
    System.out.println("Holder Name : " + holderName);
    System.out.println("Account Type : " + accountType);
    System.out.printf("Balance : Rs.%.2f%n", balance);
}
}

// Savings Account class - inherits from Account
class SavingsAccount extends Account {
    private double interestRate;

    SavingsAccount(String accountNumber, String holderName,
        double balance, double interestRate) {
        super(accountNumber, holderName, balance, "Savings");
        this.interestRate = interestRate;
    }

    // Calculate interest based on balance
    public double calculateInterest() {
        return balance * interestRate / 100;
    }

    @Override
    public void displayAccountDetails() {
        super.displayAccountDetails();
        System.out.println("Interest Rate : " + interestRate + "%");
        System.out.printf("Interest : Rs.%.2f%n", calculateInterest());
    }
}

// Current Account class - inherits from Account
class CurrentAccount extends Account {
    private double overdraftLimit;

    CurrentAccount(String accountNumber, String holderName,
        double balance, double overdraftLimit) {
        super(accountNumber, holderName, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    // Overridden withdraw() - allows withdrawal up to overdraft limit
    @Override
    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance + overdraftLimit) {
            balance -= amount;
            System.out.println("Withdrawn: Rs." + amount);
            return true;
        }
        System.out.println("Withdrawal exceeds overdraft limit.");
        return false;
    }

    @Override
    public void displayAccountDetails() {
        super.displayAccountDetails();
    }
}

```

```

        System.out.printf("Overdraft Limit: Rs.%.2f%n", overdraftLimit);
    }
}

// Main class
public class BankAccountObservationDemo {
    public static void main(String[] args) {

        // Creating objects
        SavingsAccount savings =
            new SavingsAccount("SA101", "Rahul", 10000, 5.0);

        CurrentAccount current =
            new CurrentAccount("CA201", "Priya", 15000, 5000);

        System.out.println("==== SAVINGS ACCOUNT (Before Transactions) =====");
        savings.displayAccountDetails();

        System.out.println("\n==== CURRENT ACCOUNT (Before Transactions) =====");
        current.displayAccountDetails();

        System.out.println("\nDepositing Rs.2000 into Savings...");
        savings.deposit(2000);

        System.out.println("\nWithdrawing Rs.1500 from Savings...");
        savings.withdraw(1500);

        System.out.println("\nWithdrawing Rs.18000 from Current (overdraft test)...");
        current.withdraw(18000);

        System.out.println("\nTransferring Rs.2000 from Savings to Current...");
        savings.transfer(current, 2000);

        System.out.println("\n==== SAVINGS ACCOUNT (After Transactions) =====");
        savings.displayAccountDetails();

        System.out.println("\n==== CURRENT ACCOUNT (After Transactions) =====");
        current.displayAccountDetails();

        System.out.println("\n==== FINAL BALANCES =====");
        System.out.printf("Savings Account Final Balance : Rs.%.2f%n", savings.balance);
        System.out.printf("Current Account Final Balance : Rs.%.2f%n", current.balance);
    }
}

```

■ Compilation & Execution Commands

Save the source code exactly as:

```
BankAccountObservationDemo.java
```

Compile the program:

```
javac BankAccountObservationDemo.java
```

Run the compiled program:

```
java BankAccountObservationDemo
```

■ Output Screen

```

==== SAVINGS ACCOUNT (Before Transactions) =====
Account Number : SA101
Holder Name    : Rahul
Account Type   : Savings
Balance        : Rs.10000.00
Interest Rate  : 5.0%
Interest       : Rs.500.00

==== CURRENT ACCOUNT (Before Transactions) =====
Account Number : CA201
Holder Name    : Priya
Account Type   : Current
Balance        : Rs.15000.00
Overdraft Limit: Rs.5000.00

```

```

Depositing Rs.2000 into Savings...
Deposited: Rs.2000.0

Withdrawing Rs.1500 from Savings...
Withdrawn: Rs.1500.0

Withdrawing Rs.18000 from Current (overdraft test)...
Withdrawn: Rs.18000.0

Transferring Rs.2000 from Savings to Current...
Withdrawn: Rs.2000.0
Deposited: Rs.2000.0
Transferred: Rs.2000.0 to CA201

===== SAVINGS ACCOUNT (After Transactions) =====
Account Number : SA101
Holder Name    : Rahul
Account Type   : Savings
Balance        : Rs.8500.00
Interest Rate  : 5.0%
Interest       : Rs.425.00

===== CURRENT ACCOUNT (After Transactions) =====
Account Number : CA201
Holder Name    : Priya
Account Type   : Current
Balance        : Rs.-1000.00
Overdraft Limit: Rs.5000.00

===== FINAL BALANCES =====
Savings Account Final Balance : Rs.8500.00
Current Account Final Balance : Rs.-1000.00

```

```

C:\Users\toad\I\BankAccountObservationDemo (Java Application) C:\Users\toad\I\
Transferring Rs.2000 from Savings to Current...
Withdrawn: Rs.2000.0
Deposited: Rs.2000.0
Transferred: Rs.2000.0 to CA201

===== SAVINGS ACCOUNT (After Transactions) =====
Account Number : SA101
Holder Name    : Rahul
Account Type   : Savings
Balance        : Rs.8500.00
Interest Rate  : 5.0%
Interest       : Rs.425.00

===== CURRENT ACCOUNT (After Transactions) =====
Account Number : CA201
Holder Name    : Priya
Account Type   : Current
Balance        : Rs.-1000.00
Overdraft Limit: Rs.5000.00

===== FINAL BALANCES =====
Savings Account Final Balance : Rs.8500.00
Current Account Final Balance : Rs.-1000.00

```

Fig: Console output of BankAccountObservationDemo in Eclipse IDE

■ Result

The Bank Account Management System was successfully implemented in Java using classes, objects, and inheritance. The program demonstrates constructors, deposits, withdrawals (including overdraft handling via method overriding in CurrentAccount), interest calculation, and fund transfers between SavingsAccount and CurrentAccount objects.